

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1153

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and pattern access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 30%

Total Marks:10

QUESTIONS: 1

Annexure A

Reverse Engineering Task

Code of Conduct:

I K Madan Mohan Reddy (192411206) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Kyreddy Madan

Annexure A

Reverse Engineering Task

1. An existing system is developed without following object-oriented principles, resulting in tightly coupled classes and poor modularity.

Question:

Reverse engineer the system by applying object-oriented design principles (encapsulation, inheritance, polymorphism) and restructure it into a modular design. Explain your approach.

2. A legacy application does not use any design patterns, leading to repetitive code and difficulty in extending features.

Question:

Analyze and refactor the system by introducing appropriate GoF design patterns, and justify how they improve flexibility and maintainability.

3. A system combines user interface, business logic, and database operations in a single layer.

Question:

Reconstruct the system into a layered architecture (View, Business, Access layers) and explain how responsibilities are distributed across layers.

4. An application directly accesses the database from multiple classes, causing redundancy and tight coupling.

Question:

Reverse engineer the system by introducing a Data Access Layer (DAO pattern) and explain how it improves integration with OODBMS.

5. A poorly designed system lacks clear object responsibilities, causing confusion and overlapping functionalities.

Question:

Identify and redefine object responsibilities using GRASP principles, and redesign the system for better clarity and maintainability.

6. A legacy system tightly binds object creation logic within multiple classes, making it difficult to manage dependencies and extend functionality.

Question:

Reverse engineer the system by applying appropriate creational design patterns (such as Factory Method or Abstract Factory) to decouple object creation from usage. Explain how your redesign improves scalability, flexibility, and code reusability.

RUBRICS FOR CALCULATION**Reverse Engineering Task**

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Question:

GRASP-Based Responsibility Redesign

A poorly designed system lacks clear object responsibilities, causing confusion and overlapping functionalities. Multiple classes perform similar tasks, no single class owns a complete responsibility, and tracing a feature from end to end requires inspecting many unrelated classes. This leads to bugs that are difficult to locate and fix.

Answer:

GRASP-Based Responsibility Redesign

Introduction

GRASP (General Responsibility Assignment Software Patterns) is a set of object-oriented design principles used to assign responsibilities to classes and objects in a software system. GRASP helps developers create systems that are maintainable, reusable, scalable, and loosely coupled. In an e-commerce shopping platform, applying GRASP principles improves the overall architecture by assigning responsibilities to the most suitable classes.

The major GRASP principles include:

- Information Expert
- Creator
- Controller
- Low Coupling
- High Cohesion
- Polymorphism
- Pure Fabrication
- Indirection
- Protected Variations

Information Expert

Definition

The responsibility should be assigned to the class that has the necessary information to fulfill it.

Application in E-Commerce System

Product Class

The Product class contains product details such as price, stock quantity, and description.

Benefit

- Responsibilities are assigned logically.
- Reduced complexity.
- Easier maintenance.

Problems in the Existing System

The poorly designed system exhibits the following issues:

- **Responsibility Confusion**
- Multiple classes handle the same functionality, making it unclear which class is responsible for a particular task.
- **Code Duplication**
- Similar methods and logic appear in different classes, increasing maintenance effort.
- **Tight Coupling**
- Classes depend heavily on each other, so changes in one class affect many others.
- **Low Cohesion**
- A single class performs unrelated functions, violating the principle of focused responsibility.
- **Difficult Debugging**
- Finding the source of a bug requires checking multiple classes because responsibilities are scattered.
- **Poor Scalability**
- Adding new features often requires modifying many existing classes, increasing complexity.

Creator

Principle

Assign object creation responsibility to a class that closely uses or contains the object.

Problem

Many unrelated classes create Order Item objects.

Benefit

- Controlled object creation.
- Better organization.
- Reduced confusion.

Low Coupling

Principle

Minimize dependencies between classes.

Problem

Classes directly call many other classes.

Benefit

- Easier modifications.
- Reduced impact of changes.
- Better flexibility.

High Cohesion

Principle

A class should have closely related responsibilities.

Problem

One class handles orders, payments, inventory, and reporting.

Benefit

- Easier understanding.
- Better code organization.
- Improved maintainability.

Polymorphism

Principle

Different behaviors should be implemented through interfaces and inheritance.

Problem

Large if-else statements determine payment methods.

Benefit

- Easy feature extension.
- Cleaner code.
- Better flexibility.

Pure Fabrication**Principle**

Create service classes to improve cohesion and reduce coupling.

Problem

Database operations are mixed with business logic.

Benefit

- Business logic remains clean.
- Easier database maintenance.
- Improved modularity.

Problems in the Existing System

The existing system lacks a clear distribution of responsibilities among classes. Several classes perform duplicate operations, resulting in code redundancy and inconsistency. Since responsibilities are scattered, developers often struggle to identify the correct location for implementing changes or fixing defects. The system also suffers from tight coupling, where classes depend heavily on one another. Any modification in one class may require changes in multiple dependent classes, increasing maintenance costs and the risk of introducing new bugs. Furthermore, the lack of cohesion causes classes to handle multiple unrelated tasks, making the code difficult to understand and reuse.

Applying GRASP Principles

GRASP (General Responsibility Assignment Software Patterns) provides a set of guidelines that help assign responsibilities to classes in a structured manner. By following these principles, the system can be redesigned to improve maintainability, scalability, and flexibility. The redesign focuses on assigning responsibilities to the most suitable classes and reducing unnecessary dependencies.

Information Expert Principle

The Information Expert principle suggests assigning responsibilities to the class that possesses the necessary information to perform them. For example, an Order class should calculate the total order amount because it contains information about products, quantities, and prices. Instead

of spreading calculation logic across multiple classes, the responsibility is centralized within the Order class. This improves clarity, reduces duplication, and ensures that related functionality remains together.

Creator Principle

The Creator principle states that a class should be responsible for creating objects that it closely uses or contains. For instance, an Order class should create OrderItem objects because it manages and stores them. This approach keeps object creation organized and prevents unrelated classes from taking responsibility for creating objects they do not manage. As a result, the system becomes more intuitive and easier to maintain.

Controller Principle

The Controller principle introduces a dedicated controller class to handle requests from the user interface. Rather than allowing interface components to directly interact with business logic and database classes, controllers receive requests and coordinate the appropriate operations. This separation of concerns improves code organization and ensures that presentation logic remains independent of business rules and data access operations.

Achieving Low Coupling

Low coupling is essential for building maintainable software systems. In the redesigned architecture, dependencies between classes are minimized by using interfaces and service layers. Classes communicate through well-defined contracts rather than direct implementations. This reduces the impact of changes and allows individual components to evolve independently. Low coupling also simplifies testing because classes can be tested in isolation.

Maintaining High Cohesion

High cohesion ensures that a class focuses on a specific set of closely related responsibilities. Instead of creating large classes that handle multiple tasks, responsibilities are divided among specialized classes such as OrderService, PaymentService, InventoryService, and CustomerService. Each class performs a single purpose effectively, improving readability and simplifying maintenance.

Using Polymorphism for Flexibility

Polymorphism enables different classes to implement the same behavior in their own way. For example, different payment methods such as Credit Card, UPI, and Net Banking can implement a common Payment interface. This eliminates lengthy conditional statements and makes it easier to introduce new payment methods without modifying existing code. The system becomes more flexible and easier to extend.

Pure Fabrication for Better Design

Sometimes additional classes are created solely to improve system design. These classes, known as Pure Fabrication classes, do not represent real-world entities but provide useful services. Examples include DAO classes, repository classes, and utility services. By moving database operations into DAO classes, business objects remain focused on business logic rather than technical implementation details.

Indirection for Reduced Dependencies

Indirection introduces intermediate layers between system components. Instead of allowing the presentation layer to communicate directly with the database, requests pass through controllers, service classes, and data access layers. This separation reduces direct dependencies and makes the architecture more flexible. Changes in one layer can be implemented without significantly affecting other layers.

Protected Variations Against Future Changes

Software systems often need to adapt to changing business requirements and external technologies. The Protected Variations principle uses interfaces and abstractions to isolate areas that are likely to change. For example, different payment gateways can implement a common PaymentGateway interface. If a new gateway is introduced, only a new implementation is added without affecting the rest of the system.

Benefits of the Redesigned System

The GRASP-based redesign offers numerous advantages. Responsibilities become clearly defined, making the system easier to understand. Code duplication is significantly reduced because each responsibility is assigned to a single appropriate class. The architecture promotes high cohesion and low coupling, resulting in better maintainability and scalability. Debugging becomes easier because developers can quickly identify the class responsible for a particular functionality. Additionally, the system becomes more flexible and adaptable to future enhancements.

Conclusion

A poorly designed system with overlapping responsibilities, code duplication, and tight coupling creates significant challenges for development and maintenance. By applying GRASP principles such as Information Expert, Creator, Controller, Low Coupling, High Cohesion, Polymorphism, Pure Fabrication, Indirection, and Protected Variations, responsibilities can be assigned more effectively. The redesigned system becomes more structured, maintainable, scalable, and easier to understand. As a result, software quality improves, development effort decreases, and the system becomes better prepared to accommodate future